

Assignment 1

Task 1 — Database design

a. Add more attributes to the given relations (bill, customer, partner).

Answer:

Customer

- customer_id
- first_name
- last_name
- email
- phone_number
- street
- house_number
- city
- country_code

Partner

- partner_id
- company_name
- country_code
- partner_email
- partner_phone

Bill

- bill_id
- branch_id
- bill_amount
- bill_date

We added descriptive attributes to the tables by considering what would actually be required in practice. The bill needs to have an ID and an amount but also an issue date at which it was issued. For partners and customers, we also thought about the information the company would most likely require to know who they are and to reach out to them, for example, email, telephone, and location.

b. Add a minimum of two relations that represent entities. Make sure that there is at least one many-to-many relationship between the entities.

Answer:

We added the following entities:

- country(country_code, country_name)
- branch(branch_id, branch_name, country_code, branch_email, phone_number)
- product(product_id, product_name, product_price, product_category, product_rating)

And the following junction tables:

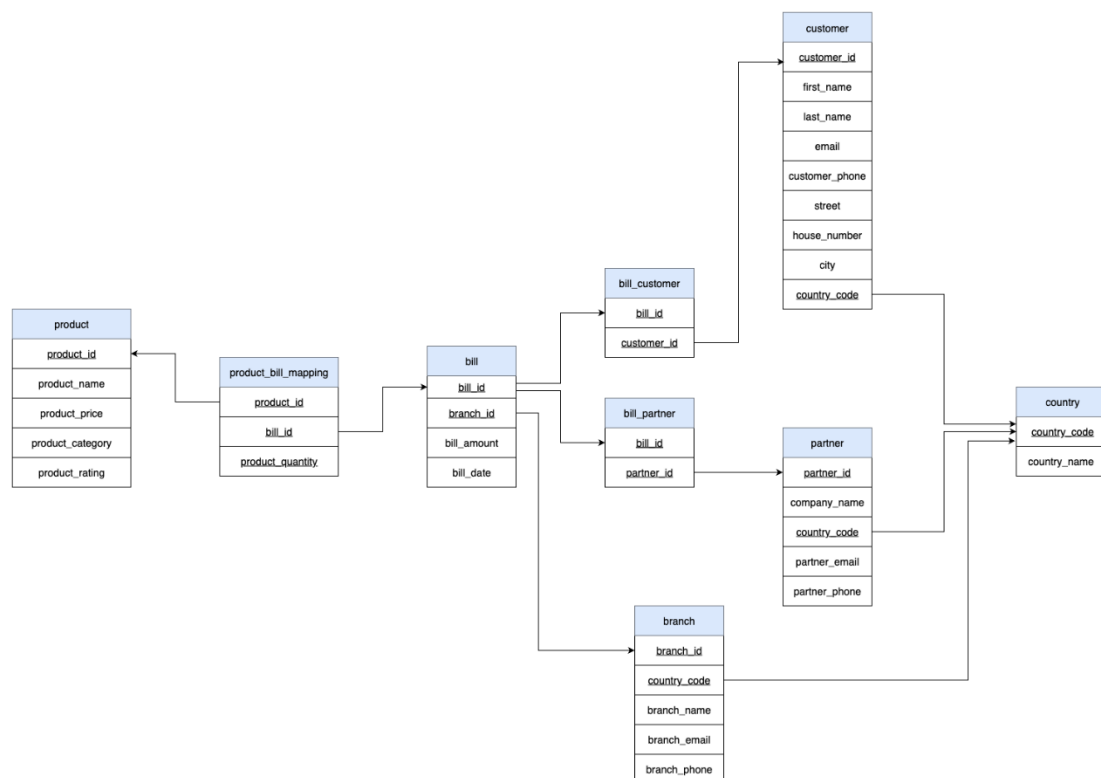
- product_bill_mapping(bill_id, product_id, product_quantity)
- bill_customer(bill_id, customer_id)
- bill_partner(bill_id, partner_id)

When studying schemas, we asked which of these would be necessary for a trading company. We concluded that countries, branches, and products would be necessary. We built junction tables to model many-to-many relationships such as bills and products, partner, and clients.

c. Draw the schema chart for the database.

Answer:

The schema chart is provided in the diagram.



We began with the main entities (bill, customer, partner) and later added supporting ones like country, branch, and product. Next, we mapped their relationships countries link to customers, partners, and branches, bills link to branches, products, customers, and partners through many to many connections.

d. Identify the cardinalities of the relationships between the entities and explain how you can represent the relationships in your design.

Answer:

- country, customer: one-to-many
- country, branch: one-to-many
- country, partner: one-to-many
- branch, bill: one-to-many
- bill, product: many-to-many (through product_bill_mapping)
- bill, customer: many-to-one (through bill_customer)
- bill, partner: many-to-one (through bill_partner)

We looked at each relationship and asked: can one item be linked to many others? If yes, we checked if it also works the other way around. If both sides can have many, we used a many-to-many relationship with a junction table. If only one side can have many, we used a one-to-many with a foreign key.

e. List all the primary-key-foreign-key constraints in the database.

Answer:

Primary keys:

- country(country_code)
- branch(branch_id)
- customer(customer_id)
- partner(partner_id)
- product(product_id)
- bill(bill_id)
- product_bill_mapping(bill_id, product_id, product_quantity)
- bill_customer(bill_id, customer_id)
- bill_partner(bill_id, partner_id)

Foreign keys:

- branch.country_code → country.country_code
- customer.country_code → country.country_code
- partner.country_code → country.country_code
- bill.branch_id → branch.branch_id
- product_bill_mapping.bill_id → bill.bill_id
- product_bill_mapping.product_id → product.product_id
- bill_customer.bill_id → bill.bill_id
- bill_customer.customer_id → customer.customer_id
- bill_partner.bill_id → bill.bill_id
- bill_partner.partner_id → partner.partner_id

Each table needs a primary key to tell its rows apart. For simple tables like customer or product, we just used an ID. For junction tables, we used two IDs together as a key because both are needed to show the connection. When one table points to another, we added a foreign key.

f. Write the SQL code that creates the whole database in SQLite and save the file as .sql that can be run directly from the sqlite3 bash using the command `.read my_file.sql`

Answer:

We started by turning on foreign key support using `PRAGMA foreign_keys = ON` so that the links between the tables would work properly.

Then, we created the main tables: `country`, `customer`, `partner`, `branch`, `product`, and `bill`. Each table has a primary key, usually just an ID, to make sure each row is unique. For the linking tables like `bill_customer`, `bill_partner`, and `product_bill_mapping`, we used two columns as the primary key because they connect two things.

To make sure the data is correct, we added CHECK rules. We used foreign keys for one-to-many connections, and junction tables for many-to-many relationships.

At the end, we added sample data to all tables to test our queries and make sure everything works as expected.

g. Pick one of the relations (tables) and explain if it is in the BCNF or not (justify your answer).

Answer:

We chose the relation `customer(customer_id, first_name, last_name, email, phone_number, street, house_number, city, country_code)`.

The functional dependency is:

`customer_id` → `first_name`, `last_name`, `email`, `phone_number`, `street`, `house_number`, `city`, `country_code`

This means that the `customer_id` decides everything else in the row. Since `customer_id` is the primary key and it's the only thing that determines the rest, all other fields depend directly on it.

Conclusion: The table is in BCNF because the only determinant is a key.

Task 2 — Querying the database

a. Joining more than two tables

Natural language

Find all bills together with the customer name and the branch name.

Relational Algebra

$$\begin{aligned} &\pi_{\text{bill_id}, \text{bill_date}, \text{first_name}, \text{last_name}, \text{branch_name}} (\\ &(\text{bill} \bowtie_{\text{bill.bill_id} = \text{bill_customer.bill_id}} \text{bill_customer}) \\ &\bowtie_{\text{bill_customer.customer_id} = \text{customer.customer_id}} \text{customer} \\ &\bowtie_{\text{bill.branch_id} = \text{branch.branch_id}} \text{branch}) \end{aligned}$$

The steps we followed were:

First, we looked at the natural language query and understood what information we needed. The result should include:

- From bill: bill_id, bill_date
- From customer: first_name, last_name
- From branch: branch_name

Next, we checked which tables contained this information. We needed the bill table (bills), the customer table (customer names), and the branch table (branch names). We also needed the bill_customer table, because it connects bills and customers as a junction table in a many-to-many relationship.

After that, we joined the tables to connect the data. First, we joined bill with bill_customer on bill_id (to see which customers are linked to each bill). Then, we joined the result with the customer table on customer_id. Finally, we joined the bill table with the branch table on branch_id to get the branch name.

In the end, we used projection to keep only the columns we were asked for.

SQL

(SQL script)

b. Aggregate function

Natural language

Find all bills together with the customer name and the branch name.

Relational Algebra

$$\pi_{\text{bill_id, bill_date, first_name, last_name, branch_name}} ($$

$$(\text{bill} \bowtie_{\text{bill.bill_id} = \text{bill_customer.bill_id}} \text{bill_customer})$$

$$\bowtie_{\text{bill_customer.customer_id} = \text{customer.customer_id}} \text{customer}$$

$$\bowtie_{\text{bill.branch_id} = \text{branch.branch_id}} \text{branch})$$

The steps we followed were:

First, we read the natural language and saw that we need: the bill info, the customer name, and the branch name.

Then, we checked which tables give us this data: bills are in the bill table, customer names are in the customer table, and branch names are in the branch table. To link bills and customers, we also need the bill_customer table.

After that, we joined these tables using their matching IDs (bill_id, customer_id, branch_id) so we could bring all the information together.

Finally, we projected only the columns that we need: bill_id, bill_date, first_name, last_name, and branch_name.

SQL

(See SCL script)

c. Nested query

Natural language

Find customers who have spent more than the average bill amount.

Relational Algebra

1. Compute how much each customer spent in total → Customers are connected to bills through the bill_customer table. Each bill has an amount (bill.bill_amount). To find how much a customer has spent overall, we need to sum all the amounts of their bills.

$$T = \gamma_{\text{customer_id}; \text{SUM}(\text{bill.bill_amount}) \rightarrow \text{total_spent}}$$

$$(\text{customer} \bowtie_{\text{customer.customer_id} = \text{bill_customer.customer_id}} \text{bill_customer} \bowtie_{\text{bill_customer.bill_id} = \text{bill.bill_id}} \text{bill})$$

2. Now we calculate the average value of all bills.

$$A = \gamma; \text{AVG}(\text{bill_amount}) \rightarrow \text{avg_bill}(\text{bill})$$

3. Compare (1) with (2) → Finally, we compare each customer's total spending with the average bill amount. We take the Cartesian product $T \times A$, which adds the average bill

value to every row of T, and then we keep only the rows where total_spent is greater than avg_bill.

Result = σ total_spent > avg_bill (T $\times$ A)

SQL

(See SQL script)

Task 3: Data extraction and entity resolution using Python

- a. Write a Python code that allows you to connect to the database file, send SQL queries to the database and extract the results.

Answer

```
#Importing necessary libraries
import sqlite3
import pandas as pd
import os

#Connect to the database file, send SQL queries to the database and extract the results.

mydbpath = '/content/Assignment1_DB_finalised.db' #Path to the database file
#Establishing a connection to the database
if os.path.exists(mydbpath):
    try:
        conn = sqlite3.connect(mydbpath)
        cursor = conn.cursor()
        print("Database connection successful.")
    except sqlite3.Error as e:
        print(f"SQLite error: {e}")
else:
    print("Database file does not exist. Connection aborted.")

country_data = cursor.execute("SELECT * FROM country").fetchall()
bill_data = cursor.execute("SELECT * FROM bill").fetchall()
partner_data = cursor.execute("SELECT * FROM partner").fetchall()
customer_data = cursor.execute("SELECT * FROM customer").fetchall()

# Printing the results
print("Country Table:")
for row in country_data:
    print(row)

print("\nBill Table:")
for row in bill_data:
    print(row)

print("\nPartner Table:")
for row in partner_data:
    print(row)

print("\nCustomer Table:")
for row in customer_data:
    print(row)
```

Sqlite3, pandas and os libraries were used to connect to the the database, execute queries and store the results and pandas dataframe.

First, the database was created in DB Browser sqlite, saved locally and uploaded to colab. The 'mydbpath' variable stores the path to db which is used with the connect method of sqlite3 to connect to the database. Thereafter, using the execute method 4 select queries were run and the fetched results were stores in country_data, bill_data, partner_data and customer_data variables. Finally, for loops were applied to each of the variables to parse and print the relations from the sql queries.

- b. Read the content of the customer relation (table) into Pandas DataFrame.

Answer

```
[3]: #Read the content of the customer relation (table) into Pandas DataFrame.

customer_df = pd.read_sql_query("SELECT * FROM customer", conn)
customer_df
```

- c. Using a similarity function that compares two records (similar to the one in the tutorial), report the customers with similarity > 0.7.

Using Jaro similarity:

Answer

```
#Using jaro similarity function that compares two records report the customers with similarity > 0.7.

def jaro(s, t):
    '''Jaro similarity between two strings.'''
    s_len = len(s)
    t_len = len(t)

    if s_len == 0 and t_len == 0:
        return 1

    match_distance = (max(s_len, t_len) // 2) - 1

    s_matches = [False] * s_len
    t_matches = [False] * t_len

    matches = 0
    transpositions = 0

    for i in range(s_len):
        start = max(0, i - match_distance)
        end = min(i + match_distance + 1, t_len)

        for j in range(start, end):
            if t_matches[j]:
                continue
            if s[i] != t[j]:
                continue
            s_matches[i] = True
            t_matches[j] = True
            matches += 1
            break

    if matches == 0:
        return 0

    k = 0
    for i in range(s_len):
        if not s_matches[i]:
            continue
        while not t_matches[k]:
            k += 1
        if s[i] != t[k]:
            transpositions += 1
        k += 1

    return ((matches / s_len) +
            (matches / t_len) +
            ((matches - transpositions) / 2) / matches) / 3

jaro(customer_df['first_name'][0], customer_df['first_name'][19]) #Comparing 'Anna' and 'Sanna' from customer
```

The `jaro(s, t)` is an implementation of the Jaro similarity function, where 1 means identical and 0 means no similarity between the two strings. The algorithm looks at how many characters match between two strings and how many of those matches are out of order, then it combines the three terms. `s_len` and `t_len` represent the length of the strings, `match_distance` defines how far apart the characters are allowed to be and still match, `s_matches` and `t_matches` mark which positions `s` and `t` have been matched.

The first for loop is used to find the matched characters and the second for loop counts the transpositions among matched characters. Lastly, the return statement outputs the proportions of `s` and `t` and how well the matched characters line up in order.

Using Jaccard similarity for comparing records:

$$\text{Jaccard Similarity } J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

```
#Using jaccard similarity function that compares two records and reports the customers with similarity > 0.7.

def jaccard(s, t):
    '''Compute Jaccard similarity between two strings.'''
    set_s = set(s.lower())
    set_t = set(t.lower())
    intersection = set_s & set_t
    union = set_s | set_t
    if not union:
        return 0
    return len(intersection) / len(union)
```

The `jaccard(s, t)` implements Jaccard similarity by first converting the record into a set of unique characters to remove duplicates, then converts the characters to lower case and finds the intersection and union of the two sets and stores them in the `intersection` and `union` variables respectively. 1 implies they are the same and 0 implies the records share nothing in common. This is done with the help of set logic in Python (& and |). The function loops through the records of the customer table and computes and compares the Jaccard similarity by taking the average similarity score across all the columns. The threshold is set to > 0.7 such that if the similarity value is greater than this, the records are considered to be similar. These are then stored in a list and printed.

	Customer 1 ID	Customer 2 ID	Average Similarity
0	C00006	C00021	0.742
1	C00007	C00022	0.853
2	C00008	C00023	0.732